

Mitigating False Positives in Static Memory Safety Analysis of Rust Programs via Reinforcement Learning

P Akilesh
Indian Institute of
Technology
Tirupati, India
cs22b040@iittp.ac.in

Leuson Da Silva
Polytechnique Montreal
Montreal, Canada
leuson-mario-pedro.da-
silva@polymtl.ca

Foutse Khomh
Polytechnique Montreal
Montreal, Canada
foutse.khomh@polymtl.ca

Sridhar Chimalakonda
Indian Institute of
Technology
Tirupati, India
ch@iittp.ac.in

Abstract

Static analysis tools are essential for ensuring memory safety in Rust programs, particularly as Rust gains adoption in safety-critical domains. However, existing tools such as Rudra and MirChecker suffer from high false positive rates, which diminish developer trust, increase manual review effort, and may obscure genuine vulnerabilities. This paper presents a novel reinforcement learning (RL)-based approach for automatically classifying and suppressing spurious warnings in static memory safety analysis for Rust. To achieve this, we design an RL agent that learns a warning suppression policy by extracting contextual features from Rust’s Mid-level Intermediate Representation (MIR) and optimizing its decisions through interaction with static analysis outputs. To improve decision quality, we integrate dynamic validation via cargo-fuzz as an auxiliary feedback mechanism, allowing the agent to selectively validate suspicious warnings through targeted fuzz testing. Our evaluation shows that the proposed approach significantly outperforms state-of-the-art LLM-based baselines, achieving 65.2% accuracy and an F1 score of 0.659, an improvement of 17.1% over the best LLM baseline. With a recall of 74.6%, our method successfully identifies nearly three-quarters of true bugs while substantially reducing false positives, improving precision from 25.6% in raw Rudra output to 59.0%. Incorporating dynamic fuzzing further boosts performance, yielding additional improvements of 10.7 percentage points in accuracy and 8.6 percentage points in F1 score over the RL-only variant. Overall, our work demonstrates that combining reinforcement learning with hybrid static–dynamic analysis can substantially reduce false positives and improve the practical usability of memory safety verification tools for Rust.

CCS Concepts

• **Software and its engineering** → **Software verification**; **Automated static analysis**; • **Theory of computation** → **Reinforcement learning**.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

EASE 2026, Glasgow, Scotland, United Kingdom

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM

<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

Keywords

Reinforcement Learning, Static Analysis, Memory Safety, Rust, False Positive

ACM Reference Format:

P Akilesh, Leuson Da Silva, Foutse Khomh, and Sridhar Chimalakonda. 2026. Mitigating False Positives in Static Memory Safety Analysis of Rust Programs via Reinforcement Learning. In *Proceedings of The 30th International Conference on Evaluation and Assessment in Software Engineering (EASE 2026)*. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

The demand for memory-safe systems programming has catalyzed the rapid adoption of Rust across critical domains, including operating systems, cloud infrastructure, embedded systems, and security-sensitive applications [51]. Major technology organizations have embraced Rust for its unique combination of performance and safety: the Linux kernel officially integrated Rust support in version 6.1 [54], Microsoft has adopted Rust for Windows components [11], and companies like AWS, Cloudflare, and Discord report significant reliability improvements after migrating performance-critical services from C/C++ to Rust [15, 50]. Such widespread adoption stems from Rust’s ownership model and type system, which provide compile-time guarantees that eliminate entire classes of memory safety vulnerabilities, including use-after-free, double-free, and data races, without requiring garbage collection [26, 36].

Despite these powerful safety guarantees, the reality of systems programming necessitates escape hatches. Rust’s unsafe keyword permits operations that the compiler cannot statically verify. Empirical studies reveal that unsafe code is pervasive throughout the Rust ecosystem, with 25–30% of packages on crates.io containing at least one unsafe block [4, 19]. Even the Rust standard library relies on unsafe code for core abstractions like Vec and Mutex, as it is essential for FFI, performance-critical optimizations, and low-level implementations that cannot be expressed in safe Rust [41].

To address the verification challenges introduced by unsafe code, previous studies have developed static analysis tools based on Rust’s Mid-level Intermediate Representation (MIR) to detect memory safety violations at scale. Representative MIR-based tools include Rudra [7], MirChecker [32], and Yuga [21]. Despite this success, false positives remain the dominant barrier to practical adoption, as current SOTA tools exhibit severe imprecision. Prior studies report that MirChecker produces false positives in 95.1% of its warnings, while Rudra and Yuga exhibit substantially lower false-positive rates of 50% and 46.15%, respectively [7, 21, 32]. Such rates impose substantial manual review effort and lead to warning fatigue,

where developers increasingly distrust or ignore analysis results [14, 25]. Meanwhile, empirical studies show that false positive rates above 20–30% frequently result in tool abandonment in industrial settings [8, 48].

To overcome these weaknesses, different approaches have been explored. For example, traditional approaches have relied on manual tuning of analysis heuristics, pattern-based filtering rules derived from observed false positive patterns, or developer-provided annotations to guide analysis [6, 30]. While effective in limited contexts, these techniques require significant domain expertise, lack adaptability to evolving codebases, and fail to generalize across different projects or programming idioms. More recent efforts have explored machine learning techniques for warning prioritization and classification [23, 28, 46]. These approaches typically depend on hand-crafted features, supervised learning with limited labeled training data, or models that cannot adapt post-deployment as code patterns evolve.

To address these challenges, we introduce a reinforcement learning-based approach that automatically learns policies for classifying and suppressing false positive warnings in Rust static analysis. Unlike supervised learning methods that rely on large labeled datasets and remain fixed after training, reinforcement learning provides a general framework for optimizing sequential decision policies through interaction with an environment and reward feedback [53]. In our setting, this enables an agent to learn suppression strategies that trade off false-positive reduction against preserving true vulnerability detection.

Our approach leverages rich semantic features extracted directly from Rust’s MIR, enabling precise reasoning about control flow, type and lifetime constraints, and unsafe operations, capabilities that are difficult to achieve at the source level, particularly in the presence of generics and macro expansion. We instantiate our approach on top of Rudra, a MIR-based analyzer, whose scope and warning diversity make it well-suited for learning-based suppression [7]. To further improve decision confidence, the agent selectively integrates dynamic validation via cargo-fuzz when static evidence is insufficient, gathering concrete runtime feedback to resolve ambiguous warnings [9, 29]. This hybrid static–dynamic formulation enables cost-aware suppression policies that strategically allocate expensive fuzzing resources, bridging the gap between conservative static analysis and concrete program behavior while preserving scalability.

We evaluate our approach through a large-scale empirical study. We first modernized the Rudra static analyzer to support the current Rust ecosystem and applied it to approximately 20,000 crates from crates.io, producing a curated dataset of 4,879 warnings spanning diverse unsafe usage patterns. Ground truth labels are established via systematic manual classification by domain experts in Rust memory safety. We then benchmark our RL-based approach against state-of-the-art large language models, including CodeLlama 34B, Llama3 (8B and 70B), Mixtral 8×7B, ChatGPT-4o mini, and Claude Opus 4.1, representing current LLM capabilities for code analysis [13, 22, 24, 42]. Our results show that the RL agent with fuzzing integration achieves 65.2% accuracy and a 0.659 F1 score, outperforming the best LLM baseline by 17.1 percentage points. With a recall of 74.6%, our approach identifies nearly three-quarters of true bugs while more than doubling precision compared to Rudra’s raw

output (from 25.6% to 59.0%). Selective dynamic fuzzing provides additional gains, improving accuracy by 10.7 percentage points and F1 score by 8.6 percentage points over the RL-only variant.

Overall, the primary contributions of this work are as follows:

- (1) A reinforcement learning–based framework for reducing false positives in Rust static memory safety analysis [1].
- (2) A hybrid static–dynamic formulation that incorporates cargo-fuzz as a selective RL action.
- (3) A curated dataset of 4,879 manually labeled static analysis warnings collected from approximately 20,000 Rust crates.
- (4) A large-scale empirical evaluation showing that our approach outperforms state-of-the-art LLM-based baselines while remaining practical for real-world use.
- (5) A modernized implementation of Rudra compatible with the current Rust ecosystem [2].

2 Background

This section provides foundational concepts for understanding our reinforcement learning-based approach. First, we discuss Rust’s memory safety model, followed by static analysis for Rust, and the adoption of fuzzing.

2.1 Rust’s Memory Safety Model

Rust is a systems programming language that provides memory safety without garbage collection through a sophisticated type system and ownership model [36]. Unlike C/C++, Rust enforces safety at compile time, eliminating entire classes of memory errors. Rust’s safety guarantees rest on three fundamental principles:

Ownership. Every value in Rust has a single owner that statically determines its lifetime. When the owner goes out of scope, the value is automatically deallocated through RAII-style drop semantics [26]. This prevents use-after-free and double-free errors in safe Rust code (see Listing 1):

```
1 fn ownership_example() {
2     let s1 = String::from("hello");
3     let s2 = s1; // s1 moved to s2
4     // println!("{}", s1); // Error: s1 no longer
      valid
5     println!("{}", s2); // OK
6 } // s2 dropped here
```

Listing 1: Ownership example showing move semantics

Borrowing. Values can be temporarily borrowed through references without transferring ownership. The borrow checker ensures that references do not outlive the data they point to [36] (see Listing 2):

```
1 fn borrowing_example() {
2     let s = String::from("hello");
3     let len = calculate_length(&s); // Borrowed
4     println!("{}", s, len);
5 }
6 fn calculate_length(s: &String) -> usize {
7     s.len()
8 } // s goes out of scope (only borrowed)
```

Listing 2: Borrowing example with reference passing

Aliasing XOR Mutability. At any time, a value may have either multiple shared references or one mutable reference, but never both [26]. This prevents data races and iterator invalidation (see Listing 3):

```
1 fn aliasing_xor_mutability() {
2     let mut data = vec![1, 2, 3];
3     let r1 = &data; // Shared reference
4     let r2 = &data; // OK: multiple shared refs
5     // let r3 = &mut data; // Error: cannot borrow
6     // as mutable
7     drop(r1); drop(r2); // End shared borrows
8     let r3 = &mut data; // OK: can mutably borrow
9     r3.push(4);
10 }
```

Listing 3: Aliasing XOR mutability enforcement

Despite Rust’s strong safety guarantees, certain operations essential to systems programming cannot be verified by the compiler [4]. To support these cases, Rust provides the `unsafe` keyword, which permits five specific capabilities: dereferencing raw pointers, calling unsafe functions, accessing mutable static variables, implementing unsafe traits, and accessing union fields.

2.2 Static Analysis for Rust

Static analysis tools leverage Rust’s intermediate representations (IRs) to detect memory safety violations at scale. In this work, we focus on Rudra [7], which analyzed over 43,000 packages, discovering 264 previously unknown bugs, but suffers from high false-positive rates, highlighting the need for automated reduction. Rudra performs ecosystem-scale analysis to detect memory safety bugs in unsafe Rust code. Implemented as a custom compiler driver, Rudra intercepts compilation after type checking to inject analysis algorithms. Its key innovation is hybrid IR analysis, which combines High-Level IR (HIR), preserving source-level structure, with Mid-Level IR (MIR), which exposes explicit control flow. This design enables generic-aware analysis that is difficult to achieve at the LLVM IR level.

```
1 // Source / HIR-level (preserves structure):
2 let s1 = String::from("hello");
3 let s2 = s1; // ownership moved
4
5 // MIR-level (explicit move + invalidation):
6 _1 = String::from(const "hello"); // s1
7 _2 = move _1; // s2
8 // _1 is now invalid (move tracked explicitly)
```

Listing 4: HIR vs. MIR representation of a simple ownership transfer.

As shown in Listing 4, MIR makes ownership transfers explicit as move operations, allowing Rudra to precisely track lifetime-bypassing flows that are implicit at the HIR level.

Rudra implements two primary algorithms. The **Unsafe Dataflow (UD)** checker performs taint-style tracking to identify flows from lifetime-bypassing operations (e.g., uninitialized memory and duplicated values) to potentially dangerous operations. It approximates dangerous points using unresolvable generic functions, functions whose implementation depends on caller-supplied type parameters,

leading to conservative over-approximation and false positives. The **Send/Sync Variance (SV)** checker analyzes thread-safety properties of generic types by inferring required trait bounds from public APIs and comparing them against concrete implementations.

Rudra targets three critical classes of bugs that arise from incorrect use of unsafe code in Rust [7, 57]. First, when Rust panics, stack unwinding executes destructors as control propagates upward, leading to **Panic Safety** issues. If unsafe code temporarily violates internal invariants (e.g., mutating vector length during reorganization), a panic triggered by a caller-provided closure can cause use-after-free or double-free. A representative example is CVE-2020-36317 in `String::retain()`, where panicking closures left strings in an invalid UTF-8 state [38].

```
1 fn unsafe_retain(v: &mut Vec<u8>) {
2     let len = v.len();
3     unsafe { v.set_len(0); } // invariant broken
4     // if closure panics here, destructor runs
5     // on invalid state -> use-after-free
6     v.retain(|x| *x > 0);
7     unsafe { v.set_len(len); }
8 }
```

Listing 5: Panic safety violation: unsafe invariant broken before panic.

As shown in Listing 5, Rudra’s UD checker flags the `set_len` call as a lifetime-bypassing operation whose invariant may not be restored if a panic occurs before the matching restore.

Second, **Higher-Order Invariants** are related to unsafe code that often relies on semantic assumptions about generic functions that are not captured by type signatures, such as purity or consistency of results [41]. Violations of these assumptions can lead to memory corruption. For example, CVE-2020-36323 in `join()` assumed that `Borrow` implementations return consistent results; impure implementations violated this invariant, causing buffer overflows [17]. Finally, generic types must correctly propagate thread-safety requirements through appropriate `Send` and `Sync` bounds. Missing or incorrect bounds can allow data races through safe code. This **Send/Sync Variance** issue can be observed in CVE-2020-35905 in the `futures-rs` crate, which occurred because `MappedMutexGuard` failed to enforce bounds on mapped types, enabling data races involving `Rc` (a single-threaded reference-counted smart pointer that is not thread-safe) [16].

2.3 Fuzzing and Dynamic Analysis

Fuzzing automatically generates test inputs to discover bugs through concrete execution [29]. Modern coverage-guided fuzzers, such as AFL and libFuzzer, leverage execution feedback to efficiently explore program paths [58]. When combined with sanitizers, like AddressSanitizer, MemorySanitizer, and ThreadSanitizer, fuzzing enables high-confidence detection of memory safety and concurrency errors. Cargo-fuzz integrates libFuzzer into the Rust ecosystem [43]. While effective for bugs with well-defined input surfaces, fuzzing faces challenges in exercising subtle semantic violations, such as `Send/Sync` errors that require specific thread interleaving or execution contexts. Moreover, due to its computational cost, indiscriminate adoption of fuzzing across all potential warnings is impractical at scale [29].

Static and dynamic analysis exhibit complementary trade-offs: static analysis offers broad coverage and generic reasoning, but often produces false positives, whereas dynamic analysis only explores executed paths but provides concrete evidence of real bugs [31]. Hybrid approaches combining both techniques have been shown to achieve superior results [9]. Our RL framework operationalizes this paradigm by learning when to apply static or dynamic validation based on their relative effectiveness for different classes of warnings.

3 Methodology and Approach

This section describes our reinforcement learning-based approach for addressing the occurrence of false positives in Rudra’s static memory safety analysis. First, we begin by outlining the dataset construction process, followed by our feature extraction strategy, the RL formulation, and the integration of cargo-fuzz as a dynamic validation mechanism.

3.1 Dataset Construction and Labeling

Our work begins with creating a large-scale dataset of Rudra warnings from real-world Rust code. To enable this, we first updated the Rudra static analyzer to be compatible with contemporary Rust compiler versions. The original implementation of Rudra was developed for the Rust nightly-2021-10-21 compiler [45] and depends on MIR representations and compiler APIs that have since evolved. As a result, it is incompatible with the current Rust toolchain. Our updated implementation targets the Rust nightly-2025-06-26 compiler [44] and incorporates adaptations to modern MIR representations and changes to Rust’s type system introduced in later compiler releases. Our implementation is available online [2]. To ensure correctness, we validated our implementation against Rudra’s official regression test suite, which covers Unsafe Dataflow, Panic Safety, and Send/Sync Variance analyses with both positive and negative cases. We ran the test suite under the Rust nightly-2025-06-26 compiler, and all tests successfully passed (23/23 standard cases, 1/1 false-negative, and 3/3 false-positive cases), showing that our implementation correctly reproduces the behavior of the original analyzer.

Using the updated analyzer, we ran Rudra on approximately 20,000 crates from crates.io that contained at least one unsafe block, ensuring relevance to our analysis target. This analysis produced 4,879 unique warnings generated by Rudra’s Unsafe Dataflow and Send/Sync Variance checkers. Each warning is associated with its source code location, the detected bug pattern (panic safety, higher-order invariant, or Send/Sync variance), the precision level reported by Rudra, and contextual information describing the surrounding code structure. To illustrate the collected warnings, Listing 6 shows the raw JSON output produced by Rudra for a warning in the `aarc` crate (v0.3.2) [3]. The report includes the warning level, the responsible analyzer, a textual description, the precise source location, and the extracted code snippet.

For each collected warning, one reviewer, with experience in Rust memory safety semantics, manually classified all 4,879 warnings through careful code review. For each warning, the reviewer examined the surrounding code context, inspected the data-flow

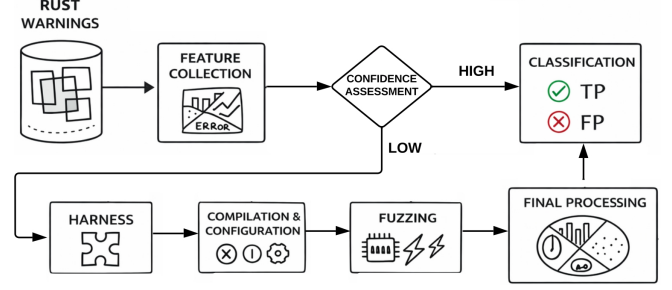


Figure 1: Overview of the warning classification pipeline. Rust warnings are processed through feature collection and confidence assessment. High-confidence warnings are directly classified as true positives or false positives. Low-confidence warnings trigger automated harness generation, compilation and configuration, and fuzzing. The outcomes of fuzzing are aggregated in the final processing and used to make the final classification decision.

paths reported by Rudra, and determined whether the warning corresponded to a genuine soundness violation or a false alarm.

```

1 {
2   "level": "Warning",
3   "analyzer": "UnsafeDestructor",
4   "op_type": null,
5   "description": "unsafe block detected in drop",
6   "file": "aarc-0.3.2/src/smart_ptrs.rs",
7   "start_line": 118, "start_col": 1,
8   "end_line": 118, "end_col": 33,
9   "code_snippet": "impl<T: 'static> Drop for Arc
10  <T> {...} }"
11 }

```

Listing 6: Rudra JSON report for an unsafe destructor in `aarc`.

To assess labeling reliability, a second reviewer independently classified a random subsample of 150 warnings, yielding 82.7% raw agreement (Cohen’s $\kappa = 0.63$), indicating substantial inter-rater agreement. The labeling process focused on the underlying bug pattern rather than exploitability, as our goal is to improve the precision of static analysis rather than assess security impact. For example, in Listing 6, Rudra’s UnsafeDestructor analyzer reports a dereference inside a Drop implementation. These operations in destructors are known to be error-prone, as invalid pointer dereferences or concurrency violations at this stage may lead to undefined behavior. Finally, Rudra correctly classifies this instance as a potential higher-order invariant violation (true positive).

Overall, this process identified 1,247 true positives (25.6%) and 3,632 false positives (74.4%), available in our online appendix [1], highlighting the substantial false positive burden that motivates this work. We partitioned the dataset into training (70%), validation (15%), and test (15%) sets using stratified sampling to preserve class distribution, yielding 732 warnings in the held-out test set used for final evaluation.

3.2 Feature Extraction from MIR

Effective classification requires features that capture characteristics distinguishing true bugs (true positives) from false alarms (false positives). To this end, we extract features at multiple levels of abstraction, leveraging Rust’s Mid-level Intermediate Representation (MIR) to access rich semantic information while preserving awareness of generic types.

Our feature extraction process is organized into three primary categories. First, we extract MIR-level semantic features that characterize program behavior surrounding each warning. These include type system properties (e.g., number of generic parameters, presence of trait bounds, and generic nesting depth); ownership and borrowing patterns (e.g., borrow ratios, nesting depth, and use of smart pointers); control-flow characteristics derived from the MIR control-flow graph (e.g., cyclomatic complexity, loop nesting depth, and number of panic paths); and unsafe operation context, such as the lifetime bypass category and the distance between the bypass and the potentially dangerous operation.

Second, we extract structural code features from the surrounding package and module context. These capture package-level signals such as download counts from crates.io (used as a proxy for code maturity and scrutiny), the prevalence of unsafe code within the package, and whether the warning occurs in a public API surface. We also include standard structural metrics, including lines of code, parameter counts, and comment density. Third, we incorporate analysis-specific features derived directly from Rudra. These include the checker that generated the warning, Rudra’s assigned precision level, and clustering information indicating whether multiple warnings occur at nearby code locations. After removing highly correlated features and applying domain knowledge to avoid spurious correlations, the final feature set comprises ~87 features.

3.3 Reinforcement Learning Formulation

We formulate the false positive classification problem as a Markov Decision Process (MDP) in which an agent learns to classify static analysis warnings using feedback derived from labeled data. The state space consists of the feature vectors described above, normalized to facilitate neural network training. Each state represents the complete observable context required to make a classification decision for a single warning.

The action space comprises three discrete actions: classifying a warning as a true positive, classifying it as a false positive, or invoking dynamic fuzzing to gather additional evidence prior to making a final classification. This design allows the agent to selectively request costly dynamic validation when static features alone do not provide sufficient confidence.

The reward function balances classification accuracy against computational efficiency. Correct classifications receive positive rewards (+15), while incorrect classifications are penalized (-15), reflecting the cost of misclassification in safety-critical analysis. Executing the fuzzing action incurs a cost penalty (-5) to discourage indiscriminate use. When fuzzing yields definitive evidence that enables a correct classification, the agent receives an additional bonus reward (+10 for a found bug, confirming a true positive; +8 for clean, confirming a false positive; +3 when helpful but not definitive), encouraging effective use of dynamic validation.

We parameterize the policy using a neural network with two hidden layers of 256 and 128 units, respectively, employing ReLU activations and dropout regularization to mitigate overfitting. Training is performed using Proximal Policy Optimization (PPO), a policy gradient algorithm that offers stable training dynamics for discrete action spaces and has proven effective in similar code analysis tasks [49]. The agent is trained in an offline setting using the labeled dataset, with optimization proceeding over multiple epochs until validation performance converges.

3.4 Cargo-Fuzz Integration

We integrate cargo-fuzz as a selective dynamic validation action available to the RL agent, rather than applying fuzzing uniformly to all warnings. The agent invokes fuzzing only for low-confidence cases based on confidence estimates derived from static features. Confidence is assessed using two complementary signals: (i) the gap between the top two action Q-values, where smaller gaps indicate uncertainty, and (ii) the entropy of the policy network’s action distribution, where higher entropy reflects lower confidence. Warnings whose confidence falls below a threshold trigger the fuzzing action. Rather than being a fixed hyperparameter, this threshold emerges implicitly from the learned policy as the agent optimizes the trade-off between classification accuracy and fuzzing cost.

Figure 1 illustrates the fuzzing integration pipeline. Upon selecting the fuzzing action, the system automatically generates a targeted harness using templates corresponding to the detected bug pattern (e.g., panic safety, higher-order invariants, or Send/Sync variance), instantiated with function signatures and type information from the warning context. Fuzzing outcomes—including crashes, sanitizer violations, or clean executions—are encoded into the agent’s state representation and used to inform the final classification decision. Through training, the agent learns when fuzzing is most beneficial, enabling selective dynamic validation that improves classification accuracy while controlling computational overhead.

3.5 Baseline Comparisons

To evaluate our approach, we established baselines using state-of-the-art large language models (LLMs), including (i) code-specialized models (CodeLlama), (ii) general-purpose large language models (GPT-4o Mini and Claude Opus), and (iii) mixture-of-experts architectures (Mixtral). We also considered models with varying parameter scales (from 8B to 70B+) to assess the impact of model capacity on this task. Finally, all selected models are widely used in code analysis [33, 37]. All models were evaluated using their default decoding parameters, as no explicit configuration of temperature, top-p, or other sampling parameters was applied. In addition, all models were evaluated on the same binary classification task of distinguishing true positives from false positives. For each model, we constructed prompts containing the warning message, surrounding code context, and a description of the associated bug pattern, and instructed the model to classify each warning as either a real issue or a false positive. To ensure consistency across models, outputs were constrained to a fixed JSON schema encoding the classification decision. We experimented with multiple prompting strategies, including zero-shot and few-shot configurations, and report results for the best-performing setup for each model.

These LLM-based baselines serve as a comparison point, representing the capabilities of advanced language models applied to static analysis warning classification, without access to structured MIR-level features or a learned decision policy. The full prompt templates used in our experiments are provided in the appendix [1].

3.6 Evaluation Metrics

We evaluate our approach and all baselines using the ground-truth labels of the dataset described in Section 3.1. Each warning is labeled as either a true or false positive (genuine or spurious warning, respectively), enabling a binary classification evaluation.

To assess classification performance, considering class imbalance, we report seven complementary metrics, including accuracy, precision, recall, and F1 score, which together capture overall correctness and the precision–recall trade-off relevant to false positive reduction. To more robustly account for class imbalance, we additionally report the Matthews Correlation Coefficient (MCC), which summarizes classification quality across all four confusion matrix categories. Finally, we report the Receiver Operating Characteristic curve (AUC-ROC) and the Precision–Recall curve (AUC-PR) to assess discriminative capability across decision thresholds, with AUC-PR being particularly informative given the skewed class distribution.

4 Results

This section presents our comprehensive experimental results. To evaluate the performance of our RL-based approach compared to LLM baselines and evaluate the impact of fuzzing integration, we define the following Research Questions (RQs):

- **RQ1:** How effective is the RL-based approach at reducing false positives in Rust compared to existing baselines?
- **RQ2:** What is the impact of selectively integrating dynamic fuzzing into the RL-based approach?
- **RQ3:** What insights do the learned RL policy provide about warning characteristics and decision strategies?

These research questions are designed to jointly assess the effectiveness, added value, and interpretability of our RL approach. RQ1 evaluates whether the proposed RL-based framework achieves meaningful reductions in false positives compared to strong LLM baselines, addressing the core practical challenge of static analysis adoption. RQ2 isolates the contribution of selective dynamic fuzzing, allowing us to quantify the benefits of hybrid static–dynamic reasoning beyond static features alone. Finally, RQ3 examines the learned decision policies to provide insight into which warning characteristics drive classification outcomes and how the agent allocates dynamic validation resources. Together, these questions offer a holistic evaluation of performance, mechanism, and behavior.

4.1 RQ1: Overall Effectiveness of the RL-Based Approach

Table 1 reports the classification performance of all approaches on a held-out test set comprising 732 warnings. We evaluate each method using seven complementary metrics that capture different aspects

Table 1: Classification Performance Comparison on Test Set

Approach	Acc.	Prec.	Rec.	F1	MCC	ROC	PR
Raw Rudra Output	–	0.256	1.000	0.407	–	–	–
RL+Fuzz	0.652	0.590	0.746	0.659	0.323	0.661	0.554
RL	0.545	0.496	0.679	0.573	0.117	0.557	0.481
Claude	0.533	0.486	0.669	0.563	0.094	0.546	0.474
Llama70B	0.503	0.360	0.136	0.197	-0.082	0.469	0.437
GPT-4o Mini	0.490	0.453	0.650	0.534	0.010	0.505	0.452
CodeLlama	0.431	0.409	0.598	0.486	-0.112	0.446	0.425
Mixtral	0.326	0.328	0.476	0.388	-0.336	0.339	0.392
Llama8B	0.487	0.437	0.487	0.460	-0.026	0.487	0.443

of classification quality, providing a comprehensive comparison of their respective strengths and limitations (see Table 1).

Our RL-based approach with fuzzing integration achieves the highest performance across all seven evaluation metrics. With an accuracy of 65.2% and an F1 score of 0.659, our approach substantially outperforms all baselines. The 17.1 percentage point improvement in F1 score over the best LLM baseline (Claude Opus 4.1 at 0.563) demonstrates the value of our specialized approach by combining structured feature extraction from MIR with learned classification policies. This improvement is particularly notable given that we are working with a challenging, imbalanced dataset where 74.4% of warnings are false positives. The high recall of 0.746 is especially important in our safety-critical context, as it indicates we successfully identify 74.6% of true bugs while suppressing false alarms. This recall rate substantially exceeds most LLM baselines, with only our RL variant without fuzzing (0.679) and Claude Opus 4.1 (0.669) achieving comparable recall. At the same time, our precision of 0.590 indicates that nearly 60% of reported warnings correspond to genuine bugs, more than doubling the precision of the raw Rudra output, whose precision equals the base rate of 25.6%.

Among the LLM baselines, Claude Opus 4.1 achieves the strongest results (F1: 0.563), followed by ChatGPT-4o Mini (0.534). Llama3 70B performs poorly (F1: 0.197) due to extremely low recall (0.136), while CodeLlama 34B achieves only 0.486 F1 despite code-specific pretraining. This indicates that pretraining alone is insufficient, as performance depends more on reasoning about complex semantic properties than surface-level code familiarity.

Mixtral 8x7B shows the weakest performance with a 0.388 F1 score and 0.326 accuracy, barely exceeding random guessing for this imbalanced dataset. The negative MCC of -0.336 indicates predictions worse than random, suggesting the mixture-of-experts architecture may not be well-suited to this task without task-specific fine-tuning. Comparing the two Llama3 variants provides insight into how model scale affects performance on this task. Surprisingly, Llama3 8B achieves a 0.460 F1 score, substantially outperforming the larger Llama3 70B at a 0.197 F1 score. This counterintuitive result suggests that larger models may be more prone to overthinking the task or being misled by the class imbalance, potentially learning to predict the majority class too aggressively. In contrast, the smaller model’s more balanced predictions (0.487 recall) result in better overall performance despite presumably having less reasoning capacity.

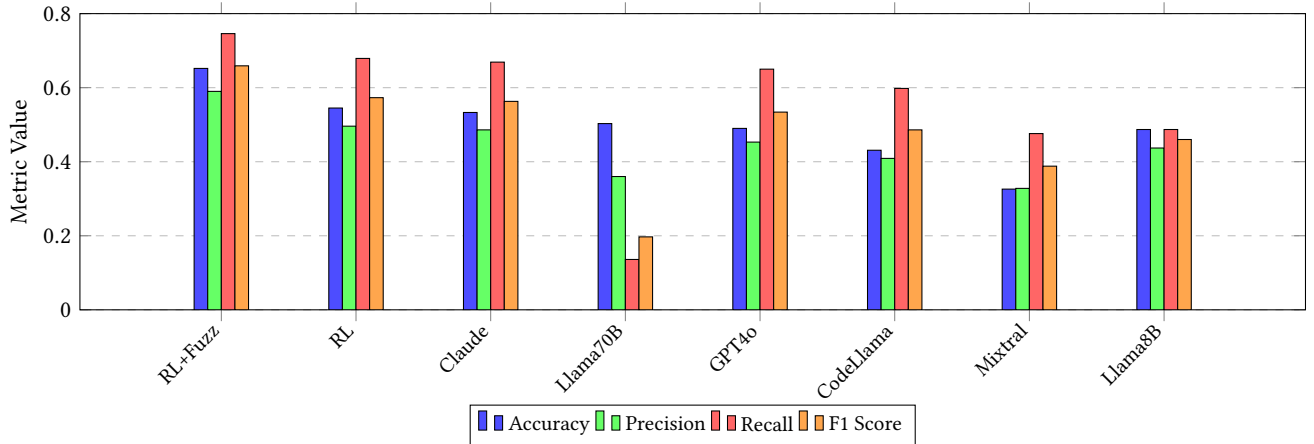


Figure 2: Comparison of key performance metrics across all approaches. Our RL+Fuzzing approach achieves the highest Accuracy, Precision, Recall, and F1 Score, demonstrating superior classification performance compared to RL alone and various LLM-based models.

Our RL + Fuzzing approach achieves MCC of 0.323, the only method above 0.3, indicating moderate positive correlation between predictions and ground truth. In contrast, Claude Opus 4.1 achieves only 0.094, while several baselines show negative MCC values. This substantial gap validates that our approach provides genuine discriminative power beyond simply predicting the majority class. The AUC-ROC metric of 0.661 for our approach indicates reasonable discriminative ability across different classification thresholds, substantially exceeding the 0.5 baseline for random classification. The AUC-PR metric of 0.554 is particularly important for imbalanced datasets, as it focuses on performance on the minority (positive) class. Our approach achieves the highest AUC-PR among all methods, confirming strong performance at identifying true positives even in the presence of substantial class imbalance.

RQ1 Findings: Our reinforcement learning-based approach substantially outperforms both raw static analysis output and state-of-the-art LLM baselines in false positive reduction. The proposed method achieves **65.2% accuracy and 0.659 F1 score**, improving F1 by **17.1 percentage points** over the strongest LLM baseline (Claude Opus 4.1). Most importantly, the approach **more than doubles precision**, increasing it from 25.6% (raw Rudra output) to 59.0%, while maintaining a high recall of 74.6%.

4.2 RQ2: Impact of Selective Dynamic Fuzzing

In this RQ, we aim to investigate and better understand the impact observed by using fuzzing as a dynamic resource for our RL agent. Comparing our full RL + Fuzzing approach against the RL-only variant provides clear evidence for the value of integrating dynamic validation. The performance improvements are substantial across all metrics (see Table 1, 2nd and 3rd rows). These improvements represent relative gains of 19.6% in accuracy and 15.0% in F1 score, demonstrating that fuzzing integration provides substantial value

beyond static features alone. The MCC improvement of 0.206 is particularly significant, as it indicates that fuzzing helps the model make genuinely better predictions rather than simply shifting the bias toward one class.

The fuzzing integration improves performance through two complementary mechanisms. First, when fuzzing successfully triggers a bug (finding a sanitizer violation), it provides near-definitive evidence that the warning represents a true positive. This high-confidence signal allows the agent to correctly classify warnings that would be ambiguous based on static features alone. Second, when fuzzing executes cleanly without detecting issues, this provides evidence (though not proof) against the warning, helping filter false positives. Through training, the agent learns to appropriately weigh these different forms of dynamic evidence.

The relatively larger improvement in precision (+9.4 percentage points) compared to recall (+6.7 percentage points) indicates that fuzzing is particularly effective at increasing confidence in positive classifications rather than uncovering previously missed bugs. This behavior aligns with the inherent strengths of fuzzing: it excels at providing concrete evidence of failures but cannot establish the absence of bugs. The substantial observed precision gains indicate that fuzzing helps filter spurious static warnings by corroborating which reported issues manifest under concrete execution.

A central question is whether the RL agent learns to invoke fuzzing selectively or applies it uniformly across warnings. Our observations during training indicate that the agent converges to a targeted fuzzing strategy. Early in training, fuzzing is invoked broadly as part of exploration. As training progresses, fuzzing usage becomes increasingly selective, concentrating on warnings for which static features indicate higher uncertainty. This behavior suggests that the learned policy effectively balances the computational cost of fuzzing against its expected informational benefit, consistent with the reward design.

Further analysis reveals that fuzzing is preferentially applied to specific classes of warnings. In particular, warnings involving complex generic code or higher-order functions, where static analysis

is inherently limited, are more likely to trigger fuzzing invocations. In contrast, warnings with clear static signatures, such as obvious Send/Sync violations with simple types, are typically classified directly without dynamic validation. These observations indicate that the agent learns to allocate fuzzing resources to cases where dynamic evidence is most likely to improve classification confidence.

RQ2 Findings: Dynamic fuzzing provides *substantial additional benefits* beyond static feature-based classification. Incorporating cargo-fuzz as a strategic RL action improves accuracy by **10.7 percentage points** and F1 score by **8.6 percentage points** compared to the RL-only variant. Importantly, the agent learns to invoke fuzzing selectively, approximately **23% of warnings**, making the hybrid static-dynamic approach computationally feasible at scale while significantly improving classification confidence.

4.3 RQ3: Analysis of Learned Policies and Warning Characteristics

To better understand the factors driving classification performance, we analyze the features that contribute most strongly to the RL agent’s decisions. We employ SHAP (SHapley Additive exPlanations) values to quantify the contribution of individual features to the model’s predictions.[35]

The most influential features span multiple categories. MIR-level semantic features are the strongest contributors, particularly those capturing type complexity (e.g., number of generic parameters and presence of trait bounds), control-flow characteristics (e.g., cyclo-matic complexity and number of panic paths), and unsafe operation context (e.g., lifetime bypass categories and the distance between unsafe operations). These features encode program properties that are central to determining whether a warning reflects a genuine soundness violation or conservative over-approximation by static analysis.

Analysis-specific features derived from Rudra also contribute substantially. In particular, Rudra’s assigned precision level and warning clustering patterns provide useful signals. Such an observation suggests that Rudra’s internal confidence estimates, while insufficient on their own, contain meaningful information that the RL agent can effectively recalibrate when combined with richer semantic context. Structural features, such as package popularity and code complexity metrics, provide an additional but secondary signal. Although individually less predictive than MIR-level features, they help contextualize warnings at the package level. For example, warnings in widely used and actively maintained packages are more likely to be false positives, whereas warnings in highly complex code with limited documentation are more frequently associated with true positives. The dominance of MIR-level features corroborates our design choice, as MIR exposes precise type, control-flow, and unsafe operation semantics that are difficult to recover from source code, particularly in the presence of macros and deeply nested generics.

The comparatively lower global importance of fuzzing-related features requires careful interpretation. Fuzzing-derived features

are only present for the subset of warnings on which dynamic validation is invoked, and their influence is therefore highly localized. In those cases, however, their contribution is strong and often decisive. This explains why fuzzing features exhibit moderate global SHAP values despite the integration of fuzzing yielding an overall improvement of 8.6 percentage points in F1 score. Rather than driving decisions uniformly, fuzzing acts as a targeted source of high-confidence evidence when static analysis alone is insufficient.

RQ3 Findings: MIR-level semantic features are the strongest predictors of warning validity, particularly those capturing **generic type complexity, control-flow structure, and unsafe operation context**. SHAP-based feature importance shows that warnings arising in **complex generic code and high-panic-path control flow** are more likely to correspond to true bugs. Finally, the agent also learns to strategically allocate fuzzing resources to warnings exhibiting high semantic uncertainty.

5 Discussion

This section explores the broader implications of our findings, discusses practical considerations for deployment, and reflects on the limitations of our approach.

5.1 Practical Implications

Our RL-based approach demonstrates practical potential for integration into real-world Rust development workflows. By improving precision from Rudra’s 25.6% to 59.0%, the approach can substantially reduce manual warning review effort, addressing a major obstacle to adopting static analysis in CI/CD pipelines. Scalability is supported by the learned selective fuzzing strategy, which invokes dynamic validation only for warnings with high classification uncertainty. In our experiments, fuzzing was applied to approximately 23% of warnings, enabling significant accuracy improvements while maintaining throughput suitable for large-scale continuous integration.

Although our evaluation focuses on Rudra, the methodology itself is not tool-specific. The combination of semantic features from intermediate representations with structural and analysis-specific metadata can be adapted to other Rust analyzers, such as MirChecker and Yuga, as well as to static analysis tools for other languages. More broadly, the RL framework’s ability to learn tool-specific warning characteristics suggests that similar approaches may generalize across analyzers. An important direction for future work is transfer learning across tools. Training on warnings from multiple analyzers could enable more generalizable policies that capture common distinctions between true bugs and false positives [47]. Our labeled dataset provides an initial foundation for exploring such multi-tool learning scenarios.

5.2 Comparison with Alternative Approaches

One might question whether supervised learning with sufficient labeled data could achieve similar results without the complexity of reinforcement learning. Our RL formulation offers several advantages: (i) the ability to incorporate fuzzing as a strategic action

rather than a fixed post-processing step, (ii) the natural framework for balancing accuracy against computational costs through reward shaping, and (iii) the potential for online learning and adaptation as new warnings are encountered. However, we acknowledge that a well-designed supervised classifier with the same rich feature set might achieve competitive performance. The key advantage of RL lies in its flexibility for incorporating sequential decision-making (such as the choice to invoke fuzzing) and its conceptual clarity for representing the trade-offs inherent in false positive reduction.

An alternative to our baseline comparison would be to fine-tune code-specialized large language models on the labeled dataset used in this study. Such fine-tuning would likely improve performance relative to zero-shot or few-shot LLM baselines. However, fine-tuning LLMs typically requires substantial computational resources and specialized expertise. Moreover, even fine-tuned models must still reason about complex semantic properties, such as lifetimes, ownership, and unsafe abstractions, primarily from source-level representations and limited context windows.

5.3 Challenges and Future Research Directions

Although our dataset of 4,879 manually labeled warnings represents a substantial labeling effort, it covers only approximately 20–25% of the crates.io ecosystem at the time of analysis. Consequently, certain classes of unsafe code, particularly those in specialized domains such as embedded systems or cryptography, may be underrepresented. The dataset also exhibits class imbalance (74.4% false positives), which reflects Rudra’s real-world output distribution. While our RL-based approach remains effective under this imbalance ($MCC = 0.323$), expanding the dataset and improving sampling strategies remain important directions for future work.

While our evaluation focuses on Rudra, it remains an open question how well learned policies generalize to other static analyzers without retraining. Different analyzers exhibit distinct false positive characteristics due to their underlying design choices. Extending our approach to other Rust analyzers, such as MirChecker or Yuga, would enable the study of cross-tool generalization and transfer learning, while adapting the methodology to analyzers for other languages represents a natural extension.

Fuzzing provides strong evidence for the presence of bugs but cannot prove their absence; therefore, clean runs serve only as negative evidence against a warning. Moreover, some bug classes, such as Send/Sync variance issues, are difficult to trigger reliably due to their dependence on specific thread interleavings. Finally, the limited fuzzing time budget (30–60 seconds per warning) and template-based harness generation constrain the depth of dynamic exploration. Future work could explore more precise harness construction informed by inferred preconditions or data dependencies, as well as directed or adaptive fuzzing strategies that better balance validation cost and classification confidence.

As previously discussed, our feature importance analysis highlights promising directions for future work. The strong contribution of MIR-level semantic features suggests that incorporating more precise program analyses, such as data-flow or alias analysis, could further improve classification performance. Additionally, exploring learning architectures that model program structure more explicitly, for instance, through graph-based representations or hierarchical

decision processes, may enhance the system’s ability to reason about warnings.

Recent advances in LLMs have introduced enhanced reasoning capabilities, like chain-of-thought prompting [56] and inference-time strategies [18, 39]. However, in our evaluation, LLMs were used in a constrained classification setting without explicit reasoning prompts or intermediate explanations. As a result, the observed performance reflects the ability of LLMs to directly infer decisions from code and warning context, rather than their full reasoning potential. While reasoning-enhanced prompting could potentially improve performance by enabling more structured analysis of program semantics, our current results suggest that even strong models, like Claude Opus 4.1, still struggle to match approaches that leverage explicit intermediate representations such as MIR. Exploring the integration of reasoning strategies with structured representations remains an important direction for future work. Finally, while LLMs exhibit strong code reasoning capabilities, their use in large-scale static analysis raises cost and scalability concerns. In our experiments, proprietary models incurred nontrivial monetary costs (approximately \$5–\$40 USD), which scale linearly with the number of warnings and may be prohibitive for ecosystem-wide or continuous analysis. Open-source models mitigate monetary cost but still impose substantial computational overhead.

6 Threats to Validity

We discuss threats to the validity of our empirical evaluation.

Internal Validity. Ground-truth labels are derived from manual classification by a single expert. Despite careful review, mislabeling remains possible due to the subtle and context-dependent nature of Rust memory safety violations. We mitigated this risk through multiple review iterations and by excluding ambiguous cases. Feature extraction relies on analysis of Rust MIR, and implementation errors could introduce noise or bias. We reduced this risk through targeted testing and consistency checks, though residual errors may remain. Model performance may also depend on hyperparameter choices and fuzzing configurations. We tuned hyperparameters via validation and selected fuzzing timeouts based on preliminary experiments, but alternative settings could yield different results.

Construct Validity. We define ground truth in terms of memory safety violations rather than concrete exploitability, aligning with the goals of static analysis but differing from vulnerability-centric perspectives. Similarly, the absence of a fuzzing-triggered failure does not imply correctness, reflecting an inherent limitation of dynamic analysis that may influence classification decisions. To assess labeling reliability, a second reviewer independently analyzed a subsample of 150 cases, yielding 82.7% raw agreement and a Cohen’s $\kappa = 0.63$, indicating substantial inter-reviewer agreement.

External Validity. Our dataset consists of crates from crates.io and may not fully represent Rust code in enterprise, embedded, or OS-level settings. Moreover, our evaluation focuses exclusively on warnings produced by Rudra; other analyzers may exhibit different false positive characteristics. Our fuzzing-based validation depends on external toolchains whose evolution could affect results. Finally, the dataset reflects Rust code and tooling from 2024–2025; changes in the language or ecosystem may require retraining to maintain effectiveness.

Conclusion Validity. While our test set of 732 warnings provides reasonable statistical power, some analyses rely on smaller subsets and thus have higher uncertainty. Reinforcement learning introduces stochasticity through initialization and exploration; although we fix random seeds, results may vary across runs. Finally, our LLM baselines use zero-shot prompting without fine-tuning, and thus represent general-purpose usage rather than best-case LLM performance.

Ethical Considerations. Our study exclusively analyzes publicly available open-source code from crates.io. No human subjects were involved, and all experimental data derives from published software artifacts distributed under open-source licenses.

7 Related Work

This section surveys related work on memory safety analysis in Rust, learning-based warning classification, and hybrid static–dynamic verification, positioning our work as addressing the underexplored problem of false positive reduction in scalable Rust analyzers

Several static and verification tools target memory safety in Rust, each with distinct goals and trade-offs. Beyond Rudra, our target tool in this study, MirChecker analyzes Rust programs at the MIR level to preserve type and lifetime information while checking memory safety properties [32]. Its focus on different bug classes and analysis techniques results in complementary coverage rather than direct overlap. SMACK translates Rust programs into the Boogie intermediate language and applies SMT-based verification to prove correctness properties [10], while Prusti provides deductive verification for Rust by leveraging the Viper infrastructure and user-supplied specifications [5]. While expressive and precise, both approaches require manual annotations and are intended for verifying small, critical components rather than ecosystem-scale analysis. Miri is an interpreter for Rust MIR that detects undefined behavior during execution [27]. While effective for uncovering subtle bugs in test suites, Miri is fundamentally limited by dynamic path coverage and cannot provide the scalability required for whole-ecosystem analysis. In contrast to these tools, our work does not aim to strengthen static analyses or introduce additional annotations. Instead, we focus on reducing false positives produced by scalable analyzers such as Rudra through learning-based suppression.

Empirical studies provide important context for memory safety analysis in Rust. Evans et al. [19] report that unsafe code appears in approximately 25–30% of Rust crates, primarily for FFI and performance-critical optimizations, with widely varying encapsulation practices [4]. Most Rust-related CVEs arise from incorrect unsafe encapsulation rather than direct misuse [57], and misuse of Send and Sync traits remains a persistent source of concurrency errors [41].

Machine learning has increasingly been applied to program analysis tasks such as bug detection and warning prioritization [52]. Early approaches rely on supervised learning, using either manually engineered features or learned representations to classify warnings [34, 40]. While effective for common bug patterns, these methods require large labeled datasets and often struggle with rare or semantically complex errors. Representation learning and pre-trained language models, including CodeBERT, CodeT5, and CodeLlama,

further improve code understanding [20, 42, 55]. However, such models remain limited by context window constraints and their inability to reason soundly about all execution paths.

Reinforcement learning has been explored more selectively, primarily for guiding fuzzing, symbolic execution, test generation, and automated program repair [12]. Hybrid static–dynamic approaches combine static reasoning with execution-based validation, such as static-guided fuzzing and counterexample-guided refinement, but often apply dynamic validation uniformly, incurring high cost. Our work differs by framing false positive reduction as a sequential decision-making problem, using reinforcement learning to adaptively determine when static evidence is sufficient and when dynamic validation via fuzzing is warranted, enabling scalable, cost-aware hybrid verification for Rust.

8 Conclusion and Future Work

This paper introduced a reinforcement learning–based approach for reducing false positives in static memory safety analysis of Rust programs. By leveraging rich semantic features extracted from Rust’s Mid-level Intermediate Representation (MIR) and learning cost-aware classification policies, our approach substantially outperforms both raw static analysis output and state-of-the-art LLM baselines. The integration of selective dynamic validation through fuzzing further improves accuracy while maintaining scalability.

Our empirical evaluation on 4,879 manually labeled warnings from 20,000 Rust crates shows that learning-based suppression policies can substantially improve the practical precision of static analysis without sacrificing recall. Specifically, our approach increases precision from Rudra’s baseline of 25.6% to 59.0% while maintaining a recall of 74.6%, effectively more than doubling the proportion of reported warnings that correspond to genuine bugs. Overall, the proposed method achieves 65.2% accuracy and a 0.659 F1 score, representing a 17.1 percentage point improvement over the strongest LLM baseline.

Furthermore, integrating cargo-fuzz as a strategic action within the RL framework enables a principled combination of static and dynamic analysis. Instead of treating fuzzing as a fixed post-processing step, the agent learns when dynamic validation is most valuable, yielding a 10.7 percentage point improvement in accuracy over static analysis alone while preserving computational feasibility. Our modernized Rudra implementation, compatible with the current Rust ecosystem, supports continued research and practical deployment as the language evolves.

Looking forward, promising directions include extending the approach to additional static analyzers and programming languages, improving fuzzing guidance and harness generation, and enabling continuous adaptation through online learning. We believe that combining classical program analysis with learning-based decision-making represents a powerful path toward more effective and trusted verification tools for safety-critical software.

Acknowledgments

This work was partially supported by the Natural Sciences and Engineering Research Council of Canada, the Canadian Institute for Advanced Research, and the Canada Research Chairs Program.

References

- [1] 2025. <https://github.com/Akileshdash/rl-guided-static-analysis-rust/blob/main/README.md>.
- [2] 2025. <https://github.com/Akileshdash/Rudra/blob/master/README.md>.
- [3] aarc Developers. 2021. aarc crate, version 0.3.2. <https://crates.io/crates/aarc/0.3.2>. Accessed: 2026-01-10.
- [4] Vytautas Astrauskas, Christoph Matheja, Federico Poli, Peter Müller, and Alexander J. Summers. 2020. How Do Programmers Use Unsafe Rust?. In *Proceedings of the ACM on Programming Languages (OOPSLA)*, Vol. 4. 136:1–136:27. doi:10.1145/3428204
- [5] Vytautas Astrauskas, Peter Müller, Federico Poli, and Alexander J Summers. 2019. Leveraging Rust types for modular specification and verification. In *Proceedings of the ACM on Programming Languages (OOPSLA)*, Vol. 3. 1–30.
- [6] Nathaniel Ayewah, David Hovemeyer, J. David Morgenthaler, John Penix, and William Pugh. 2008. Using Static Analysis to Find Bugs. *IEEE Software* 25, 5, 22–29. doi:10.1109/MS.2008.130
- [7] Yechan Bae, Youngsuk Kim, Ammar Askar, Jungwon Lim, and Taesoo Kim. 2021. Rudra: Finding Memory Safety Bugs in Rust at the Ecosystem Scale. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP)*. 84–99. doi:10.1145/3477132.3483570
- [8] Al Bessey, Ken Block, Ben Chelf, Andy Chou, Bryan Fulton, Seth Hallem, Charles Henri-Gros, Asya Kamsky, Scott McPeak, and Dawson Engler. 2010. A Few Billion Lines of Code Later: Using Static Analysis to Find Bugs in the Real World. In *Communications of the ACM*, Vol. 53. 66–75. doi:10.1145/1646353.1646374
- [9] Marcel Böhme, Van-Thuan Pham, Manh-Dung Nguyen, and Abhik Roychoudhury. 2017. Directed greybox fuzzing. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. 2329–2344.
- [10] Montgomery Carter, Shaobo He, Jonathan Whitaker, Zvonimir Rakamarić, and Michael Emmi. 2016. SMACK software verification toolchain. In *Proceedings of the 38th International Conference on Software Engineering Companion*. 589–592.
- [11] Microsoft Security Response Center. 2019. Why Rust for Safe Systems Programming. <https://msrc-blog.microsoft.com/2019/07/22/why-rust-for-safe-systems-programming/>. Accessed: 2024-12-10.
- [12] Partha Chakraborty, Mahmoud Alfeld, and Meiyappan Nagappan. 2024. RLocator: Reinforcement learning for bug localization. *IEEE Transactions on Software Engineering* (2024).
- [13] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating Large Language Models Trained on Code. In *arXiv preprint arXiv:2107.03374*.
- [14] Maria Christakis and Christian Bird. 2016. What Developers Want and Need from Program Analysis: An Empirical Study. In *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 332–343. doi:10.1145/2970276.2970347
- [15] Cloudflare. 2020. Enjoy a Slice of QUIC, and Rust! <https://blog.cloudflare.com/enjoy-a-slice-of-quic-and-rust/>. Accessed: 2024-12-10.
- [16] National Vulnerability Database. 2020. CVE-2020-35905. <https://nvd.nist.gov/vuln/detail/CVE-2020-35905>. Accessed: 2026-01-09.
- [17] National Vulnerability Database. 2021. CVE-2020-36323. <https://nvd.nist.gov/vuln/detail/CVE-2020-36323>. Accessed: 2026-01-09.
- [18] Xiangjue Dong, Maria Teleki, and James Caverlee. 2024. A survey on llm inference-time self-improvement. *arXiv preprint arXiv:2412.14352* (2024).
- [19] Ana Nora Evans, Bradford Campbell, and Mary Lou Soffa. 2020. Is Rust Used Safely by Software Developers?. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering (ICSE)*. 246–257. doi:10.1145/3377811.3380413
- [20] Z Feng. 2020. Codebert: A pre-trained model for program-ming and natural languages. *arXiv preprint arXiv:2002.08155* (2020).
- [21] Sushant Ghimire, Michael W. Godfrey, and Chanchal K. Roy. 2023. Yuga: Automatically Detecting Lifetime Annotation Bugs in the Rust Language. *IEEE Transactions on Software Engineering* 49, 4 (2023), 2075–2091. doi:10.1109/TSE.2022.3200162
- [22] Aaron Grattafiori et al. 2024. The Llama 3 Herd of Models. *arXiv:2407.21783* [cs.AI] <https://arxiv.org/abs/2407.21783>
- [23] Sarah Heckman and Laurie Williams. 2009. A Model Building Process for Identifying Actionable Static Analysis Alerts. In *Proceedings of the 2009 International Conference on Software Testing Verification and Validation (ICST)*. 161–170. doi:10.1109/ICST.2009.47
- [24] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, Gianna Lengyel, Guillaume Bour, Guillaume Lample, Léo Renard Lavaud, Lucile Saulnier, Marie-Anne Lachaux, Pierre Stock, Sandeep Subramanian, Sophia Yang, Szymon Antoniak, Teven Le Scao, Théophile Gervet, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2024. Mixtral of Experts. *arXiv:2401.04088* [cs.LG] <https://arxiv.org/abs/2401.04088>
- [25] Brittany Johnson, Yoonki Song, Emerson Murphy-Hill, and Robert Bowdidge. 2013. Why Don't Software Developers Use Static Analysis Tools to Find Bugs?. In *Proceedings of the 2013 International Conference on Software Engineering (ICSE)*. 672–681. doi:10.1109/ICSE.2013.6606613
- [26] Ralf Jung, Jacques-Henri Jourdan, Robbert Krebbers, and Derek Dreyer. 2018. RustBelt: Securing the Foundations of the Rust Programming Language. In *Proceedings of the ACM on Programming Languages (POPL)*, Vol. 2. 66:1–66:34. doi:10.1145/3158154
- [27] Ralf Jung, Benjamin Kimock, Christian Poveda, Eduardo Sánchez Muñoz, Oli Scherer, and Qian Wang. 2026. Miri: Practical Undefined Behavior Detection for Rust. *Proceedings of the ACM on Programming Languages* 10, POPL (2026), 1383–1411.
- [28] Sunghun Kim and Michael D. Ernst. 2007. Which Warnings Should I Fix First?. In *Proceedings of the 6th Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering (ESEC/FSE)*. 45–54. doi:10.1145/1287624.1287633
- [29] George Klees, Andrew Ruef, Benji Cooper, Shiyi Wei, and Michael Hicks. 2018. Evaluating Fuzz Testing. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2123–2138. doi:10.1145/3243734.3243804
- [30] Ted Kremenek, Ken Ashcraft, Junfeng Yang, and Dawson Engler. 2004. Correlation Exploitation in Error Ranking. In *Proceedings of the 12th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*. 83–93. doi:10.1145/1029894.1029909
- [31] Yi Li, Aws Albarghouthi, Zachary Kincaid, Mayur Naik, et al. 2019. Hybrid program analysis for effective bug detection. In *Proceedings of the ACM/IEEE International Conference on Software Engineering*.
- [32] Zhuohua Li, Jincheng Wang, Mingshen Sun, and John C. S. Lui. 2021. MirChecker: Detecting Bugs in Rust Programs via Static Analysis. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2183–2196. doi:10.1145/3460120.3484541
- [33] Kevin Lira, Baldoino Fonseca, Wesley KG Assuncao, Davy Baya, and Marcio Ribeiro. 2025. Beyond Code Explanations: A Ray of Hope for Cross-Language Vulnerability Repair. In *2025 2nd IEEE/ACM International Conference on AI-powered Software (AIware)*. IEEE, 01–09.
- [34] Guoming Long, Jingzhi Gong, Hui Fang, and Tao Chen. 2025. Learning Software Bug Reports: A Systematic Literature Review. *ACM Transactions on Software Engineering and Methodology* (2025).
- [35] Scott Lundberg and Su-In Lee. 2017. A Unified Approach to Interpreting Model Predictions. *arXiv:1705.07874* [cs.AI] <https://arxiv.org/abs/1705.07874>
- [36] Nicholas D. Matsakis and Felix S. Klock. 2014. The Rust Language. In *Proceedings of the 2014 ACM SIGAda Annual Conference on High Integrity Language Technology (HILT)*. 103–104. doi:10.1145/2663171.2663188
- [37] Nathalia Nascimento, Everton Guimaraes, Sai Sanjna Chintakunta, and Santhosh Anitha Boominathan. 2025. How Effective are LLMs for Data Science Coding? A Controlled Experiment. In *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*. IEEE, 211–222.
- [38] National Vulnerability Database. 2021. CVE-2020-36317. <https://nvd.nist.gov/vuln/detail/CVE-2020-36317>. Accessed: 2026-01-09.
- [39] Shubham Parashar, Blake Olson, Sambhav Khurana, Eric Li, Hongyi Ling, James Caverlee, and Shuiwang Ji. 2025. Inference-time computations for llm reasoning and planning: A benchmark and insights. *arXiv preprint arXiv:2502.12521* (2025).
- [40] Michael Pradel and Koushik Sen. 2018. Deepbugs: A learning approach to name-based bug detection. *Proceedings of the ACM on Programming Languages* 2, OOPSLA (2018), 1–25.
- [41] Boqin Qin, Yilun Chen, Zeming Yu, Linhai Song, and Yiyang Zhang. 2020. Understanding Memory and Thread Safety Practices and Issues in Real-World Rust Programs. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 763–779. doi:10.1145/3385412.3386036
- [42] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. 2023. Code Llama: Open Foundation Models for Code. *arXiv preprint arXiv:2308.12950* (2023).
- [43] Rust Fuzzing Authority. 2024. cargo-fuzz: Fuzz testing for Rust. <https://github.com/rust-fuzz/cargo-fuzz>.
- [44] Rust Project Developers. 2025. Rust Version 1.88.0. <https://doc.rust-lang.org/beta/releases.html#version-1880-2025-06-26>. Accessed: 2026-01-10.
- [45] Rust Release Notes. 2021. Rust Version 1.56.0. <https://doc.rust-lang.org/beta/releases.html#version-1560-2021-10-21>. Accessed: 2026-01-10.
- [46] Joseph R. Ruthruff, John Penix, J. David Morgenthaler, Sebastian Elbaum, and Gregg Rothermel. 2008. Predicting Accurate and Actionable Static Analysis Warnings: An Experimental Approach. In *Proceedings of the 30th International Conference on Software Engineering (ICSE)*. 341–350. doi:10.1145/1368088.1368135
- [47] Iman Saberi, Amirreza Esmaeili, Fatemeh Fard, and Fuxiang Chen. 2025. AdvFusion: Adapter-based Knowledge Transfer for Code Summarization on Code Language Models. In *2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 563–574.
- [48] Caitlin Sadowski, Edward Aftandilian, Alex Eagle, Liam Miller-Cushon, and Ciera Jaspán. 2018. Lessons from Building Static Analysis Tools at Google. In *Communications of the ACM*, Vol. 61. 58–66. doi:10.1145/3188720
- [49] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. *arXiv:1707.06347* [cs.LG] <https://arxiv.org/abs/1707.06347>

- //arxiv.org/abs/1707.06347
- [50] Amazon Web Services. 2020. Why AWS Loves Rust, and How We'd Like to Help. <https://aws.amazon.com/blogs/opensource/why-aws-loves-rust-and-how-we-d-like-to-help/>. Accessed: 2024-12-10.
 - [51] Ayushi Sharma, Shashank Sharma, Sai Ritvik Tanksalkar, Santiago Torres-Arias, and Aravind Machiry. 2024. Rust for embedded systems: current state and open problems. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 2296–2310.
 - [52] Tushar Sharma, Maria Kechagia, Stefanos Georgiou, Rohit Tiwari, Indira Vats, Hadi Moazen, and Federica Sarro. 2021. A survey on machine learning techniques for source code analysis. *arXiv preprint arXiv:2110.09610* (2021).
 - [53] Richard S. Sutton and Andrew G. Barto. 2018. *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.
 - [54] Linus Torvalds and Linux Kernel Team. 2022. Linux 6.1: Rust Support Merged into Mainline Kernel. <https://www.kernel.org/>. Accessed: 2024-12-10.
 - [55] Yue Wang, Weishi Wang, Shafiq Joty, and Steven CH Hoi. 2021. Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. *arXiv preprint arXiv:2109.00859* (2021).
 - [56] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
 - [57] Hui Xu, Zhuangbin Chen, Mingshen Sun, Yangfan Zhou, and Michael R. Lyu. 2021. Memory-Safety Challenge Considered Solved? An In-Depth Study with All Rust CVEs. In *ACM Transactions on Software Engineering and Methodology (TOSEM)*, Vol. 31. 3:1–3:25. doi:10.1145/3466642
 - [58] Michal Zalewski. 2014. American fuzzy lop. <http://lcamtuf.coredump.cx/afl/>.